

Recursion

Introduction

Recursion is a technique in which a function solves a problem by calling itself on a smaller version of the same problem. It is one of the most elegant ideas in computer science, and also one of the most confusing for beginners.

You have seen recursion outside code. Stand between two mirrors and you see a reflection inside a reflection, repeating into the distance. Open a set of Russian nesting dolls and each doll holds a smaller copy of itself until you reach the tiniest one, which cannot be opened. That smallest doll is the key to the whole idea: every recursive process needs a point where it stops.

Anatomy of a recursive function

Every correct recursive function has two parts:

1. **Base case** — the simplest input, answered directly without another call. It is the smallest nesting doll.
2. **Recursive case** — the function breaks the problem into a smaller piece and calls itself on that piece, moving closer to the base case each time.

The classic example is the factorial. The factorial of n , written $n!$, is the product of all whole numbers from 1 to n , so $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$. Notice that $5!$ is simply $5 \times 4!$. That self similar structure makes factorial a natural fit for recursion.

```
int factorial(int n) {  
    if (n <= 1) {           // base case  
        return 1;  
    }  
    return n * factorial(n - 1); // recursive case  
}
```

Calling `factorial(5)` asks for $5 \times \text{factorial}(4)$, and so on down to `factorial(1)`, which returns 1. The results then multiply back up: 1, 2, 6, 24, 120.

A useful way to read this code is the "leap of faith": assume `factorial(n - 1)` already returns the right answer, and check only that you combine it correctly. If the base case is right and each step is right, the whole function is right. This is the same logic as proof by mathematical induction.

How it works under the hood:the call stack

Recursion feels like magic until you see the call stack. Each time a function is called, the program pushes a **stack frame** onto the stack. The frame stores that call's parameters, local variables and the point to return to.

When `factorial(5)` runs, five frames pile up, one for each value of `n` from 5 down to 1. Each frame waits, paused, for the call above it to finish. Once the base case returns, frames are popped one by one in reverse order, and each finishes its multiplication. This is the "winding" and "unwinding" of recursion.

The stack has a fixed size. If the base case is missing, or the input never shrinks toward it, frames keep piling up until memory runs out. In Java this ends with a

`StackOverflowError` . Even a correct function can overflow on very deep inputs.

Classic examples

Recursion shines whenever a problem contains smaller copies of itself.

Fibonacci numbers. Each number is the sum of the previous two: $\text{fib}(n) = \text{fib}(n - 1) + \text{fib}(n - 2)$, with $\text{fib}(0) = 0$ and $\text{fib}(1) = 1$. Elegant, but slow without care, as shown below. **Binary search.** To find a value in a sorted array, compare it with the middle element, then search only the left or right half. Each call halves the problem, so a million elements need about 20 calls.

Tree traversal. Every node's children are roots of smaller trees, so visiting every node, or every file in nested folders, takes a few lines.

Tower of Hanoi. Move the top $n - 1$ disks aside, move the largest disk, then move the $n - 1$ disks back on top. A hard puzzle becomes three lines of code.

Recursion vs iteration

Any recursive function can be rewritten as a loop, and any loop as recursion. The choice is about clarity and cost.

	Recursion	Iteration
Readability	Short and natural for trees, graphs and divide-and-conquer	Clearer for simple repetition
Memory	One stack frame per call	Usually constant extra space
Speed	Function-call overhead on every step	Generally faster

Risk	Stack overflow on deep inputs	Infinite loop if the condition is wrong
------	-------------------------------	---

Two techniques narrow the gap. **Memoization** stores results that have already been computed. Naive recursive Fibonacci recalculates the same values again and again, taking exponential time; fib(40) makes over 300 million calls. With a cache, each value is computed once and the time drops to linear.

Tail recursion places the recursive call as the function's last action. Languages like Scheme and Scala can then reuse one stack frame, matching a loop's efficiency. Java and Python do not, so deep recursion there stays risky.

Common pitfalls and tips

Missing base case. Write it first, and test inputs like 0, 1 and empty arrays. **Not shrinking the problem.** Each call must move toward the base case, or it never ends.

Repeated work. Repeated subproblems call for memoization or dynamic programming.

Tracing every call. Trust the leap of faith and check one level at a time; for small inputs, sketch a recursion tree.

Conclusion

Recursion is a way of thinking as much as a coding technique: define the simplest case, then describe how a bigger problem reduces to a smaller one. It powers searching, sorting, tree and graph algorithms, parsers and much of functional programming. Its costs are real, namely stack memory and call overhead, but memoization and careful design keep them in check. Master the base case, trust the recursive step, and problems that once looked tangled start to solve themselves.